

From the Quantum Phase estimation algorithm to the HHL and QAOA algorithms

Morten Hjorth-Jensen

FYS5419/9419: Week of April 13-17, 2026

Outline

1. Reminder from last week about the QPE algorithm
2. Discussion of the HHL algorithms, basic math and codes with examples on how to solve differential equations
3. If we get time, start discussions of the QAOA algorithm

See also separate jupyter-notebooks at <https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/pub/week12/ipynb/week12.ipynb>

Quantum Phase Estimation: Problem Statement, reminder from Lecture April 8

Suppose U is a unitary operator and $|\psi\rangle$ is one of its eigenstates:

$$U|\psi\rangle = e^{2\pi i\phi} |\psi\rangle, \quad 0 \leq \phi < 1.$$

The goal of Quantum Phase Estimation (QPE) is to estimate the phase ϕ .

Equivalently, QPE extracts the eigenvalue of U in phase form.

Why QPE Matters

QPE is central because many physical problems can be mapped to eigenvalue estimation.

Examples:

- ▶ energy estimation in quantum chemistry,
- ▶ eigenvalue problems in many-body physics,
- ▶ Hamiltonian simulation,
- ▶ Shor's algorithm and period finding.

If we can encode a Hamiltonian into a unitary, then QPE can extract its eigenvalues.

Registers Used in QPE

QPE uses two registers:

- ▶ a control register of n qubits, initialized in $|0\rangle^{\otimes n}$,
- ▶ a system register, initialized in the eigenstate $|\psi\rangle$.

So the initial state is

$$|0\rangle^{\otimes n} |\psi\rangle .$$

The control register stores the estimated phase bits.

Step 1: Create a Uniform Superposition

Apply Hadamard gates to all control qubits:

$$|0\rangle^{\otimes n} \mapsto \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} |k\rangle.$$

So the total state becomes

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} |k\rangle |\psi\rangle.$$

This prepares the control register as a coherent superposition of all binary numbers k .

Step 2: Controlled Powers of U

Now apply controlled powers of U :

$$U^{2^0}, U^{2^1}, U^{2^2}, \dots, U^{2^{n-1}}.$$

Each control qubit determines whether the corresponding power is applied.

Because $|\psi\rangle$ is an eigenstate,

$$U^{2^j} |\psi\rangle = e^{2\pi i 2^j \phi} |\psi\rangle.$$

Hence each controlled unitary writes phase information into the control register.

Step 2: State After Controlled Powers

If the control register basis state is $|k\rangle$, then the controlled operations apply U^k , giving

$$U^k |\psi\rangle = e^{2\pi i k \phi} |\psi\rangle .$$

Therefore the combined state becomes

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i k \phi} |k\rangle |\psi\rangle .$$

The system register factors out, and the control register now contains the phase information.

Key Observation

The control register is in the state

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i k \phi} |k\rangle ,$$

which is exactly a Fourier-type state.

This is why the inverse QFT is the correct next step: it converts this phase-encoded superposition into a computational basis approximation of ϕ .

Step 3: Apply the Inverse QFT

Applying QFT^{-1} to the control register gives

$$\text{QFT}^{-1} \left(\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i k \phi} |k\rangle \right).$$

If

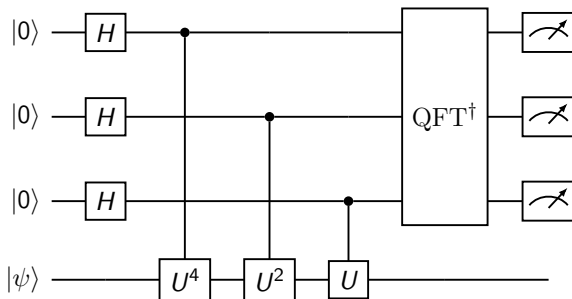
$$\phi = \frac{m}{2^n}$$

exactly for some integer m , then this becomes

$$|m\rangle.$$

So the control register stores the exact n -bit binary expansion of ϕ .

QPE Circuit



This example shows a 3-qubit control register and one system register.

Precision of QPE

With n control qubits, QPE resolves phases to approximately

$$\Delta\phi \sim \frac{1}{2^n}.$$

Thus the precision improves exponentially with the number of control qubits.

This exponential precision is one of the main reasons QPE is so powerful.

From Phases to Energies

Suppose H is a Hamiltonian with eigenstate $|E\rangle$:

$$H|E\rangle = E|E\rangle.$$

Define the time-evolution operator

$$U(t) = e^{-iHt}.$$

Then

$$U(t)|E\rangle = e^{-iEt}|E\rangle.$$

Comparing with

$$U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle,$$

we identify

$$\phi = -\frac{Et}{2\pi}.$$

Thus QPE can estimate energies if we can implement e^{-iHt} .

Implementation

A implementation of the QPE typically proceeds as follows:

1. choose the number of control qubits,
2. initialize the system register in an eigenstate or approximate eigenstate,
3. apply Hadamards to the control register,
4. apply controlled U^{2^j} ,
5. apply inverse QFT,
6. measure the control register,
7. interpret the measurement as an estimate of ϕ .

Summary of the Full Picture

$$\text{Hamiltonian } H \longrightarrow U(t) = e^{-iHt} \longrightarrow \text{QPE} \longrightarrow E$$

The QFT is the phase-decoding engine inside QPE.

VQE and QPE are complementary:

- ▶ VQE is a near-term variational energy estimator,
- ▶ QPE is a long-term high-precision eigenvalue estimator,
- ▶ VQE can provide input states for QPE.

What is the HHL algorithm?

The HHL algorithm—named after Aram Harrow, Avinatan Hassidim, and Seth Lloyd—is one of the central quantum algorithms for solving linear systems of equations.

The core idea is to solve a linear system

$$Ax = b.$$

HHL: Linear Systems Everywhere

- ▶ Solve:

$$Ax = b$$

- ▶ Appears in:
 - ▶ PDEs
 - ▶ Machine learning and quantum machine learning
 - ▶ Many-body physics
- ▶ Classical cost: $\mathcal{O}(N^3)$ (dense case)

Quantum Goal

- ▶ Encode vector as quantum state:

$$|b\rangle = \sum_i b_i |i\rangle$$

- ▶ Goal:

$$|x\rangle \propto A^{-1}|b\rangle$$

- ▶ Output is a quantum state, not a classical vector

Feynman's Vision

For an n -qubit system,

$$|\psi\rangle = \sum_{z \in \{0,1\}^n} \alpha_z |z\rangle, \quad \sum_z |\alpha_z|^2 = 1,$$

so one must track 2^n complex amplitudes.

This motivates *quantum simulation*: use controllable quantum systems to simulate other quantum systems.

Schrödinger Dynamics and Hamiltonian Simulation

The dynamics are governed by

$$i \frac{d}{dt} |\psi(t)\rangle = H |\psi(t)\rangle ,$$

with formal solution

$$|\psi(t)\rangle = e^{-iHt} |\psi(0)\rangle .$$

Hamiltonian simulation asks for a quantum circuit implementing

$$U(t) \approx e^{-iHt}$$

efficiently in the number of qubits, simulation time, and target precision.

From Simulation to Linear Systems

Many physics and numerical analysis problems reduce to

$$A\mathbf{x} = \mathbf{b}.$$

Examples include:

- ▶ finite-difference discretizations of Poisson equations,
- ▶ implicit time stepping for diffusion equations,
- ▶ linear response theory.

The Harrow–Hassidim–Lloyd (HHL) algorithm prepares

$$|x\rangle \propto A^{-1} |b\rangle.$$

Phase Estimation Setup

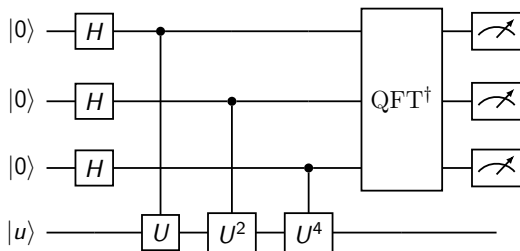
Suppose U is unitary and

$$U|u\rangle = e^{2\pi i\phi}|u\rangle, \quad \phi \in [0, 1).$$

Quantum phase estimation estimates ϕ using:

- ▶ a control register of m qubits,
- ▶ controlled powers U^{2^k} ,
- ▶ the inverse quantum Fourier transform.

Phase Estimation Circuit



How the Phases Accumulate

After the Hadamard gates, the control register is

$$\frac{1}{2^{m/2}} \sum_{k=0}^{2^m-1} |k\rangle .$$

Applying the controlled powers gives

$$\frac{1}{2^{m/2}} \sum_{k=0}^{2^m-1} e^{2\pi i k \phi} |k\rangle |u\rangle .$$

The inverse QFT maps this state approximately to

$$|\tilde{\phi}\rangle |u\rangle .$$

The Quantum Linear Systems Problem

Let

$$A|x\rangle = |b\rangle,$$

where A is Hermitian and invertible.

Write

$$A = \sum_j \lambda_j |u_j\rangle \langle u_j|, \quad |b\rangle = \sum_j \beta_j |u_j\rangle.$$

Then

$$|x\rangle \propto A^{-1} |b\rangle = \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle.$$

Main Idea Behind HHL

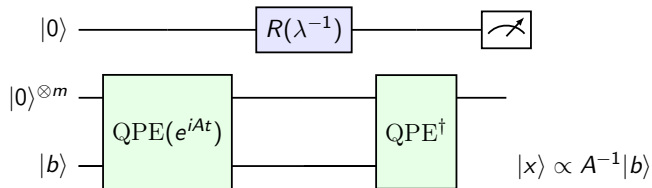
The essential map is

$$\lambda_j \mapsto \frac{1}{\lambda_j}.$$

Since this is not unitary, HHL proceeds indirectly:

1. perform phase estimation on e^{iAt} ,
2. store an approximation of λ_j ,
3. perform a controlled rotation whose amplitude depends on $1/\lambda_j$,
4. uncompute the phase register,
5. postselect on a successful ancilla measurement.

Full HHL Circuit Schematic



The Linear-System Problem

We want to solve

$$A|x\rangle = |b\rangle,$$

where A is Hermitian and invertible.

Because A is Hermitian, it has an orthonormal eigenbasis:

$$A|u_j\rangle = \lambda_j|u_j\rangle, \quad A = \sum_j \lambda_j |u_j\rangle \langle u_j|.$$

Expand the right-hand side as

$$|b\rangle = \sum_j \beta_j |u_j\rangle, \quad \beta_j = \langle u_j|b\rangle.$$

Formally,

$$A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle.$$

Why This is the Correct Formal Solution

Since

$$A = \sum_j \lambda_j |u_j\rangle \langle u_j|,$$

the inverse is

$$A^{-1} = \sum_j \frac{1}{\lambda_j} |u_j\rangle \langle u_j|,$$

provided all $\lambda_j \neq 0$.

Applying this to $|b\rangle$ gives

$$A^{-1} |b\rangle = \sum_j \frac{1}{\lambda_j} |u_j\rangle \langle u_j| \sum_k \beta_k |u_k\rangle.$$

Using orthonormality,

$$\langle u_j | u_k \rangle = \delta_{jk},$$

we get

$$A^{-1} |b\rangle = \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle.$$

So HHL must implement the spectral map

Initial State of the Algorithm

HHL uses three logical pieces:

- ▶ the *system register*, initially in $|b\rangle$,
- ▶ the *phase register*, initially in $|0\rangle^{\otimes m}$,
- ▶ one *ancilla qubit*, initially in $|0\rangle$.

So the full initial state is

$$|\Psi_0\rangle = |0\rangle_a |0\rangle_p^{\otimes m} |b\rangle_s = |0\rangle_a |0\rangle_p^{\otimes m} \sum_j \beta_j |u_j\rangle_s.$$

By linearity, the algorithm can be analyzed on each eigencomponent $|u_j\rangle$ separately.

Why Phase Estimation is Used

The inverse A^{-1} is not unitary in general, so we cannot apply it directly as a quantum gate.

Instead, define the unitary

$$U = e^{iAt}.$$

If

$$A|u_j\rangle = \lambda_j|u_j\rangle,$$

then

$$U|u_j\rangle = e^{i\lambda_j t}|u_j\rangle.$$

Comparing with the phase-estimation form

$$U|u_j\rangle = e^{2\pi i\phi_j}|u_j\rangle,$$

we identify

$$\phi_j = \frac{\lambda_j t}{2\pi}.$$

Thus quantum phase estimation converts eigenvalue information into a register that can control later operations.

State After QPE

Apply phase estimation to the phase register and system register:

$$|0\rangle^{\otimes m} |u_j\rangle \longrightarrow |\tilde{\lambda}_j\rangle |u_j\rangle ,$$

where $|\tilde{\lambda}_j\rangle$ is a binary approximation to the eigenvalue.

Therefore, by linearity,

$$|\psi_1\rangle = |0\rangle_a \sum_j \beta_j |\tilde{\lambda}_j\rangle_p |u_j\rangle_s .$$

Each eigenvector $|u_j\rangle$ is now entangled with its corresponding eigenvalue register.

Controlled Rotation

We now want an ancilla amplitude proportional to $1/\lambda_j$.
Choose a constant C such that

$$\left| \frac{C}{\lambda_j} \right| \leq 1 \quad \text{for all } j.$$

Define the controlled ancilla rotation so that

$$|\tilde{\lambda}_j\rangle |0\rangle_a \longrightarrow |\tilde{\lambda}_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} |0\rangle_a + \frac{C}{\lambda_j} |1\rangle_a \right).$$

Applying this to the entire superposition gives

$$|\Psi_2\rangle = \sum_j \beta_j |\tilde{\lambda}_j\rangle |u_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} |0\rangle_a + \frac{C}{\lambda_j} |1\rangle_a \right).$$

Rewrite by Ancilla Branches

It is useful to regroup $|\Psi_2\rangle$ according to the ancilla state:

$$|\Psi_2\rangle = |0\rangle_a \sum_j \beta_j \sqrt{1 - \frac{C^2}{\lambda_j^2}} |\tilde{\lambda}_j\rangle |u_j\rangle + |1\rangle_a \sum_j \beta_j \frac{C}{\lambda_j} |\tilde{\lambda}_j\rangle |u_j\rangle.$$

This shows explicitly:

- ▶ the undesired branch attached to $|0\rangle_a$,
- ▶ the desired inverse-weighted branch attached to $|1\rangle_a$.

Inverse QPE and Postselection

Apply the inverse QPE circuit:

$$|\tilde{\lambda}_j\rangle |u_j\rangle \mapsto |0\rangle^{\otimes m} |u_j\rangle.$$

Hence

$$|\psi_3\rangle = |0\rangle_a |0\rangle^{\otimes m} \sum_j \beta_j \sqrt{1 - \frac{C^2}{\lambda_j^2}} |u_j\rangle + |1\rangle_a |0\rangle^{\otimes m} \sum_j \beta_j \frac{C}{\lambda_j} |u_j\rangle.$$

Conditioning on ancilla outcome $|1\rangle$, the remaining system state is

$$|\tilde{x}\rangle = C \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle = C A^{-1} |b\rangle.$$

After normalization,

$$|x\rangle = \frac{A^{-1} |b\rangle}{\|A^{-1} |b\rangle\|}.$$

Success Probability

The success probability is the norm squared of the ancilla- $|1\rangle$ branch:

$$p_{\text{succ}} = \left\| C \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle \right\|^2 = C^2 \sum_j \frac{|\beta_j|^2}{\lambda_j^2}.$$

If the eigenvalues lie in

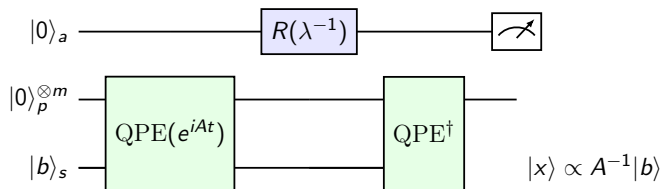
$$\lambda_j \in \left[\frac{1}{\kappa}, 1 \right],$$

and we choose $C \sim 1/\kappa$, then one typically obtains

$$p_{\text{succ}} \gtrsim \frac{1}{\kappa^2}.$$

Hence poor conditioning directly hurts the probability of successful postselection.

High-Level HHL Circuit



The three blocks correspond to:

1. spectral decomposition,
2. inverse weighting through amplitudes,
3. spectral recombination.

Circuit Step 1: QPE Block

Input:

$$|0\rangle_a |0\rangle_p^{\otimes m} \sum_j \beta_j |u_j\rangle_s.$$

After the green QPE block:

$$|0\rangle_a \sum_j \beta_j \left| \tilde{\lambda}_j \right\rangle_p |u_j\rangle_s.$$

This is the moment where the hidden spectral decomposition of $|b\rangle$ becomes explicit.

Circuit Step 2: Controlled Rotation Block

The blue block acts conditionally on the eigenvalue register:

$$|\tilde{\lambda}_j\rangle_p |0\rangle_a \mapsto |\tilde{\lambda}_j\rangle_p \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} |0\rangle_a + \frac{C}{\lambda_j} |1\rangle_a \right).$$

Hence the full state becomes

$$\sum_j \beta_j |\tilde{\lambda}_j\rangle_p |u_j\rangle_s \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} |0\rangle_a + \frac{C}{\lambda_j} |1\rangle_a \right).$$

The inverse eigenvalue weights are now encoded in ancilla amplitudes.

Circuit Step 3: Inverse-QPE Block

The inverse green block removes the eigenvalue label:

$$\left| \tilde{\lambda}_j \right\rangle_p \left| u_j \right\rangle_s \mapsto \left| 0 \right\rangle_p^{\otimes m} \left| u_j \right\rangle_s.$$

So after the inverse-QPE block:

$$\left| 0 \right\rangle_p^{\otimes m} \left[\left| 0 \right\rangle_a \sum_j \beta_j \sqrt{1 - \frac{C^2}{\lambda_j^2}} \left| u_j \right\rangle_s + \left| 1 \right\rangle_a \sum_j \beta_j \frac{C}{\lambda_j} \left| u_j \right\rangle_s \right].$$

This is why uncomputation is so important: it disentangles the phase register from the system.

Circuit Step 4: Measurement

Measure the ancilla.

If the outcome is $|1\rangle_a$, then the system is left in

$$|\tilde{x}\rangle = C \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle.$$

Normalization gives the final HHL output:

$$|x\rangle = \frac{A^{-1} |b\rangle}{\|A^{-1} |b\rangle\|}.$$

So the ancilla measurement selects the branch carrying the desired inverse spectral weights.

A Minimal Worked Example

Take

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}.$$

We choose this matrix because it is:

- ▶ Hermitian,
- ▶ positive definite,
- ▶ simple enough to diagonalize by hand.

Let the source state be

$$|b\rangle = |0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Diagonalizing the Matrix

The eigenpairs of

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$$

are

$$\lambda_1 = 4, \quad |u_1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

$$\lambda_2 = 2, \quad |u_2\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Indeed,

$$A |u_1\rangle = 4 |u_1\rangle, \quad A |u_2\rangle = 2 |u_2\rangle.$$

Expand the Source State

We now write $|b\rangle = |0\rangle$ in the eigenbasis:

$$|0\rangle = \frac{1}{\sqrt{2}} |u_1\rangle + \frac{1}{\sqrt{2}} |u_2\rangle.$$

So the coefficients are

$$\beta_1 = \beta_2 = \frac{1}{\sqrt{2}}.$$

Therefore

$$A^{-1} |b\rangle = \frac{1}{\sqrt{2}} \frac{1}{4} |u_1\rangle + \frac{1}{\sqrt{2}} \frac{1}{2} |u_2\rangle.$$

Already we see the key HHL effect: the smaller eigenvalue 2 receives the larger inverse weight $1/2$.

Exact Classical Solution in the Computational Basis

Convert back to the computational basis:

$$A^{-1} = \frac{1}{8} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix}.$$

Hence

$$A^{-1} |b\rangle = \frac{1}{8} \begin{pmatrix} 3 \\ -1 \end{pmatrix}.$$

The normalized solution state is therefore

$$|x\rangle = \frac{1}{\sqrt{10}} \begin{pmatrix} 3 \\ -1 \end{pmatrix}.$$

That is the state HHL should output after successful postselection.

How QPE Acts in This Example

Using

$$U = e^{iAt},$$

we have

$$U|u_1\rangle = e^{i4t}|u_1\rangle, \quad U|u_2\rangle = e^{i2t}|u_2\rangle.$$

Starting with

$$|0\rangle_a |0\rangle_p^{\otimes m} \left(\frac{1}{\sqrt{2}} |u_1\rangle + \frac{1}{\sqrt{2}} |u_2\rangle \right),$$

QPE yields

$$|0\rangle_a \left(\frac{1}{\sqrt{2}} |\tilde{4}\rangle_p |u_1\rangle + \frac{1}{\sqrt{2}} |\tilde{2}\rangle_p |u_2\rangle \right).$$

So the phase register now distinguishes the two eigenspaces.

How the Controlled Rotation Acts

Choose a constant $C \leq 2$, for example $C = 1$. Then

$$|\tilde{4}\rangle |0\rangle_a \mapsto |\tilde{4}\rangle \left(\sqrt{1 - \frac{1}{16}} |0\rangle_a + \frac{1}{4} |1\rangle_a \right),$$

$$|\tilde{2}\rangle |0\rangle_a \mapsto |\tilde{2}\rangle \left(\sqrt{1 - \frac{1}{4}} |0\rangle_a + \frac{1}{2} |1\rangle_a \right).$$

Thus the ancilla- $|1\rangle$ amplitudes become

$$\frac{1}{4} \quad \text{and} \quad \frac{1}{2},$$

which are precisely proportional to $1/\lambda_1$ and $1/\lambda_2$.

State Before and After Postselection

After rotation and inverse QPE, the ancilla- $|1\rangle$ branch is

$$|1\rangle_a \left(\frac{1}{\sqrt{2}} \frac{1}{4} |u_1\rangle + \frac{1}{\sqrt{2}} \frac{1}{2} |u_2\rangle \right).$$

So postselection gives the unnormalized system state

$$|\widetilde{x}\rangle = \frac{1}{4\sqrt{2}} |u_1\rangle + \frac{1}{2\sqrt{2}} |u_2\rangle.$$

This is exactly proportional to

$$A^{-1} |b\rangle.$$

Back to the Computational Basis

Using

$$|u_1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad |u_2\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix},$$

we obtain

$$|\widetilde{x}\rangle = \frac{1}{8} \begin{pmatrix} 3 \\ -1 \end{pmatrix}.$$

After normalization,

$$|x\rangle = \frac{1}{\sqrt{10}} \begin{pmatrix} 3 \\ -1 \end{pmatrix}.$$

So the whole HHL evolution reproduces the exact inverse-action of A^{-1} on $|b\rangle$.

What This Example Shows

This 2×2 example makes the logic completely explicit:

1. diagonalize A ,
2. expand $|b\rangle$ in the eigenbasis,
3. let QPE attach eigenvalue labels,
4. let the ancilla rotation multiply each eigensector by $1/\lambda_j$,
5. erase the labels with inverse QPE,
6. keep the correct branch by postselection.

That is all HHL really is: a controlled spectral inversion procedure.

Summary

The extra details added here are:

- ▶ a circuit-aligned explanation of each HHL block,
- ▶ a full derivation of the ancilla-weighting mechanism,
- ▶ a concrete 2×2 matrix example worked out from start to finish.

The essential formula remains

$$|x\rangle \propto \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle = A^{-1} |b\rangle ,$$

but now each algebraic step has a direct circuit interpretation.

Poisson Equation in One Dimension

Consider

$$-u''(x) = f(x), \quad x \in (0, 1),$$

with Dirichlet boundary conditions

$$u(0) = u(1) = 0.$$

A simple benchmark choice is

$$f(x) = \pi^2 \sin(\pi x),$$

whose exact solution is

$$u(x) = \sin(\pi x).$$

Finite-Difference Discretization

Choose interior grid points

$$x_i = ih, \quad h = \frac{1}{N+1}, \quad i = 1, \dots, N.$$

Use the centered second derivative:

$$u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}.$$

Then

$$-\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} = f_i,$$

which yields

$$A\mathbf{u} = \mathbf{b},$$

with

$$A = \frac{1}{h^2} \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & -1 & 2 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{pmatrix}.$$

A Small HHL-Compatible Example: $N = 2$

Take $N = 2$. Then

$$h = \frac{1}{3}, \quad A = 9 \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 18 & -9 \\ -9 & 18 \end{pmatrix}.$$

For $f(x) = \pi^2 \sin(\pi x)$,

$$\mathbf{b} = \pi^2 \frac{\sqrt{3}}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Exact Classical Solution for $N = 2$

The matrix

$$A = \begin{pmatrix} 18 & -9 \\ -9 & 18 \end{pmatrix}$$

has eigenpairs

$$\lambda_1 = 9, \quad |u_1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

$$\lambda_2 = 27, \quad |u_2\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Since

$$\mathbf{b} \propto \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

we have

$$A^{-1}\mathbf{b} \propto \frac{1}{9} |u_1\rangle.$$

Hence the normalized solution state is

$$|x\rangle = |u_1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Encoding the Poisson Problem on One Qubit

For $N = 2$, the solution vector has two components, so it fits in one system qubit:

$$|0\rangle \leftrightarrow \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle \leftrightarrow \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The normalized right-hand side is

$$|b\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle,$$

and the normalized exact solution state is also

$$|x\rangle = |+\rangle.$$

Pauli Representation of the Matrix

The matrix

$$A = \begin{pmatrix} 18 & -9 \\ -9 & 18 \end{pmatrix}$$

can be written as

$$A = 18I - 9X.$$

Hence

$$e^{iAt} = e^{i(18I-9X)t} = e^{i18t} e^{-i9Xt}.$$

The global phase e^{i18t} is irrelevant; the nontrivial evolution is generated by X .

Choosing the Evolution Time

Choose

$$t = \frac{\pi}{18}.$$

Then

$$\lambda_1 t = \frac{\pi}{2}, \quad \lambda_2 t = \frac{3\pi}{2},$$

so the corresponding eigenphases are well separated modulo 2π , making phase estimation straightforward in this toy model.

Worked HHL Logic for the Poisson Example

Because $|b\rangle = |u_1\rangle$ already lies in a single eigenstate of A , the HHL workflow simplifies:

1. Prepare $|b\rangle = |+\rangle$.
2. Phase estimation on e^{iAt} returns $\lambda_1 = 9$.
3. Controlled rotation multiplies the successful ancilla amplitude by $1/\lambda_1 = 1/9$.
4. Uncomputation removes the phase register.
5. Postselection returns the state $|u_1\rangle = |+\rangle$.

The overall factor $1/9$ is absorbed into normalization and the success probability.

What Observable Can We Extract?

Since HHL returns a state rather than all entries of $\mathbf{u}$, one often computes observables.

For

$$|x\rangle = |+\rangle,$$

we have

$$\langle x|X|x\rangle = 1, \quad \langle x|Z|x\rangle = 0.$$

These match the exact normalized classical solution vector

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Summary

- ▶ Hamiltonian simulation implements e^{-iHt} .
- ▶ Quantum phase estimation extracts eigenphases of a unitary.
- ▶ HHL combines Hamiltonian simulation, phase estimation, controlled rotation, uncomputation, and postselection.
- ▶ A discretized 1D Poisson equation gives a concrete and instructive worked example.

Take-Home Message

HHL is best understood as a spectral-filter algorithm for A^{-1} .

For differential equations, discretization turns the PDE into a sparse linear system, and HHL then produces a quantum state proportional to the numerical solution.

From Linear Systems to Resolvents, HHL and Green's function theory

A central observation is that HHL solves a linear system

$$Ax = b$$

by preparing a quantum state proportional to

$$|x\rangle \propto A^{-1}|b\rangle.$$

In many-body physics, the inverse of an operator is precisely the basic structure of a *resolvent* or *Green's function*.

If we define

$$G(z) = (zI - H)^{-1},$$

then solving

$$(zI - H)|x\rangle = |b\rangle$$

is equivalent to preparing

$$|x\rangle = G(z)|b\rangle.$$

Thus HHL may be viewed as a quantum algorithm for applying a resolvent operator to a source state.

Spectral Form of the Green's Function

Let

$$H|u_j\rangle = E_j|u_j\rangle.$$

Then the resolvent has the spectral decomposition

$$G(z) = (zI - H)^{-1} = \sum_j \frac{1}{z - E_j} |u_j\rangle \langle u_j|.$$

If

$$|b\rangle = \sum_j \beta_j |u_j\rangle,$$

then

$$G(z)|b\rangle = \sum_j \frac{\beta_j}{z - E_j} |u_j\rangle.$$

This is structurally identical to the HHL transformation:

$$A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle.$$

Hence, HHL implements the same spectral inversion mechanism that underlies Green's functions.

Physical Meaning of the Spectral Filter

The factor

$$\frac{1}{z - E_j}$$

acts as a spectral filter.

- ▶ Eigenmodes with energies near z are enhanced.
- ▶ Eigenmodes far from z are suppressed.
- ▶ The inverse operator encodes propagation and response properties.

In this sense, HHL is not merely a linear solver. It is a *spectral transformation algorithm*: it maps a source state to a response state through an inverse operator.

Retarded Green's Function

In condensed matter and many-body theory, one often uses

$$G^R(\omega) = \frac{1}{\omega + i\eta - H}, \quad \eta > 0.$$

The small positive η regularizes the inverse and enforces causal boundary conditions.

Applying this operator to a source state gives

$$|x(\omega)\rangle = G^R(\omega)|b\rangle.$$

Formally, an HHL-type algorithm can be interpreted as a way of preparing a quantum state proportional to this frequency-dependent response vector, provided one can encode the shifted operator

$$A(\omega) = \omega I - H$$

or

$$A(\omega) = \omega I + i\eta - H.$$

Connection to Correlation Functions

Many observables in quantum many-body physics can be written in terms of Green's functions or resolvents.

For an operator $\hat{O}$, a typical linear-response quantity has the form

$$\chi(\omega) = \langle \psi_0 | \hat{O}^\dagger \frac{1}{\omega + i\eta - (H - E_0)} \hat{O} | \psi_0 \rangle.$$

If we define the source state

$$|b\rangle = \hat{O} | \psi_0 \rangle,$$

then

$$\chi(\omega) = \langle b | \frac{1}{\omega + i\eta - (H - E_0)} | b \rangle.$$

This shows that HHL-like inversion is directly connected to computing dynamical susceptibilities.

HHL as a Green's Function Engine

At a conceptual level, HHL performs three tasks:

1. Decompose the source state into eigenmodes.
2. Invert the spectral coefficients:

$$\lambda_j \mapsto \frac{1}{\lambda_j}.$$

3. Recombine the filtered eigenmodes into the response state.

This is exactly what is needed for:

- ▶ resolvent methods,
- ▶ Green's function methods,
- ▶ frequency-domain linear response,
- ▶ projection-based many-body calculations.

Linear Response in Physics

Suppose a system with Hamiltonian H_0 is perturbed by

$$V(t) = -f(t)\hat{F}.$$

To first order, the expectation value of an observable $\hat{O}$ changes as

$$\delta\langle\hat{O}(t)\rangle = \int dt' \chi_{OF}(t-t')f(t'),$$

where χ_{OF} is the response function.

In the frequency domain, one typically writes

$$\delta\langle\hat{O}(\omega)\rangle = \chi_{OF}(\omega)f(\omega).$$

The response function is often expressible through inverse operators of the form

$$(\omega I - M)^{-1},$$

which is precisely the structure targeted by HHL.

Resolvent Form of the Susceptibility

A standard spectral representation is

$$\chi_{OF}(\omega) = \sum_{n \neq 0} \left(\frac{\langle 0 | \hat{O} | n \rangle \langle n | \hat{F} | 0 \rangle}{\omega - (E_n - E_0) + i\eta} - \frac{\langle 0 | \hat{F} | n \rangle \langle n | \hat{O} | 0 \rangle}{\omega + (E_n - E_0) + i\eta} \right).$$

This is a sum over poles, or equivalently a matrix element of a resolvent.

Thus linear response is fundamentally an *inverse-operator problem*, which places it in the same mathematical class as HHL.

What is Time-Dependent Hartree–Fock?

Time-dependent Hartree–Fock (TDHF) describes the evolution of a Slater determinant under a time-dependent mean field.

At the one-body density-matrix level,

$$i\hbar \frac{d\rho}{dt} = [h[\rho], \rho].$$

To study small oscillations around a stationary solution ρ_0 , one linearizes:

$$\rho(t) = \rho_0 + \delta\rho(t),$$

leading to an equation of motion for the fluctuation $\delta\rho$.

Linearized TDHF

After linearization, one obtains a matrix equation of the form

$$i\hbar \frac{d}{dt} \delta\rho = \mathcal{L} \delta\rho + s(t),$$

where:

- ▶ $\mathcal{L}$ is the linearized Liouvillian or RPA matrix,
- ▶ $s(t)$ is an external driving term.

In the frequency domain, this becomes

$$(\omega I - \mathcal{L}) \delta\rho(\omega) = s(\omega).$$

This is now explicitly a linear system. Its solution is

$$\delta\rho(\omega) = (\omega I - \mathcal{L})^{-1} s(\omega).$$

That is exactly the algebraic structure addressed by HHL.

TDHF as an HHL-Type Problem

The formal similarity is immediate:

$$Ax = b \quad \Longleftrightarrow \quad (\omega I - \mathcal{L})\delta\rho(\omega) = s(\omega).$$

Hence:

$$A \leftrightarrow \omega I - \mathcal{L}, \quad x \leftrightarrow \delta\rho(\omega), \quad b \leftrightarrow s(\omega).$$

If one can encode the linear-response matrix $\mathcal{L}$ into a quantum representation, then an HHL-type algorithm could in principle prepare a state proportional to the TDHF response.

Connection to Random Phase Approximation (RPA)

The small-amplitude limit of TDHF is closely related to the Random Phase Approximation (RPA).

One typically obtains a non-Hermitian eigenvalue problem of the form

$$\begin{pmatrix} A & B \\ -B^* & -A^* \end{pmatrix} \begin{pmatrix} X \\ Y \end{pmatrix} = \omega \begin{pmatrix} X \\ Y \end{pmatrix}.$$

Equivalently, one can formulate a driven linear-response problem

$$(\omega I - M) v = s.$$

Again, this is an inverse problem. The response amplitudes are obtained by applying the resolvent

$$(\omega I - M)^{-1}$$

to a source vector.

What HHL Would Compute in TDHF

In a TDHF or RPA context, one is often interested in:

- ▶ transition amplitudes,
- ▶ density oscillations,
- ▶ response to an external field,
- ▶ strength functions.

These quantities can often be expressed as matrix elements involving the response vector:

$$S(\omega) \sim \langle s | (\omega I - M)^{-1} | s \rangle.$$

Thus an HHL-type routine would not need to output the full vector $\delta\rho(\omega)$. It would already be useful if it allowed efficient estimation of such expectation values.

Strength Functions and Dynamical Response

A common observable in TDHF and RPA is the strength function

$$S_F(\omega) = -\frac{1}{\pi} \text{Im} \chi_F(\omega),$$

where

$$\chi_F(\omega) = \langle 0 | \hat{F}^\dagger \frac{1}{\omega + i\eta - (H - E_0)} \hat{F} | 0 \rangle.$$

This is exactly a Green's function expression. Hence:

TDHF/RPA response $\longleftrightarrow$ resolvent evaluation $\longleftrightarrow$ HHL-type inversion

Why This Connection is Interesting

The HHL framework is potentially relevant because:

- ▶ linear response is naturally formulated as a linear system,
- ▶ inverse operators define propagators and susceptibilities,
- ▶ many observables are expectation values rather than full vectors,
- ▶ this aligns well with quantum output models.

From a physics viewpoint, HHL is best interpreted not only as a solver of $Ax = b$, but as a quantum method for preparing *response states* and probing *spectral properties*.

Important Caveats

There are, however, important obstacles:

- ▶ The matrix A must be efficiently block-encoded or simulated.
- ▶ TDHF/RPA matrices may be non-Hermitian.
- ▶ Condition numbers may be large near resonances.
- ▶ State preparation and measurement may dominate the cost.

Therefore, the connection is mathematically deep and physically appealing, but practical advantage depends strongly on encoding, sparsity, conditioning, and the observables of interest.

Hermitian Embedding of Non-Hermitian Problems

If the response matrix M is non-Hermitian, one common strategy is Hermitian embedding:

$$\mathcal{H} = \begin{pmatrix} 0 & M \\ M^\dagger & 0 \end{pmatrix}.$$

Then $\mathcal{H}$ is Hermitian, and one may reformulate the problem in an enlarged space.

This is often the natural bridge between:

- ▶ non-Hermitian response equations,
- ▶ quantum phase estimation or HHL-type methods,
- ▶ Green's function formulations on quantum hardware.

Conceptual Summary

Key idea

HHL applies an inverse operator spectrally:

$$A^{-1} = \sum_j \frac{1}{\lambda_j} |u_j\rangle \langle u_j|.$$

Green's function viewpoint

Choose

$$A = zI - H,$$

then HHL prepares a state proportional to

$$(zI - H)^{-1} |b\rangle.$$

TDHF / linear-response viewpoint

Choose

$$A = \omega I - \mathcal{L},$$

Linear Systems in Physics

Many physics problems reduce to

$$Ax = b.$$

Examples:

- ▶ discretized Schrödinger and diffusion equations,
- ▶ linear response theory,
- ▶ Green's function methods,
- ▶ time-dependent Hartree–Fock (TDHF),
- ▶ random phase approximation (RPA).

The HHL algorithm prepares a quantum state proportional to

$$|x\rangle \propto A^{-1}|b\rangle.$$

The Core Spectral Problem

Suppose

$$A|u_j\rangle = \lambda_j|u_j\rangle, \quad |b\rangle = \sum_j \beta_j|u_j\rangle.$$

Then

$$A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j}|u_j\rangle.$$

Thus HHL implements the spectral map

$$\lambda_j \mapsto \frac{1}{\lambda_j}.$$

This is why HHL is best viewed as a *spectral inverse-operator algorithm*.

Spectral Decomposition of the Operator

Write

$$A = \sum_j \lambda_j |u_j\rangle \langle u_j|.$$

Then formally

$$A^{-1} = \sum_j \frac{1}{\lambda_j} |u_j\rangle \langle u_j|.$$

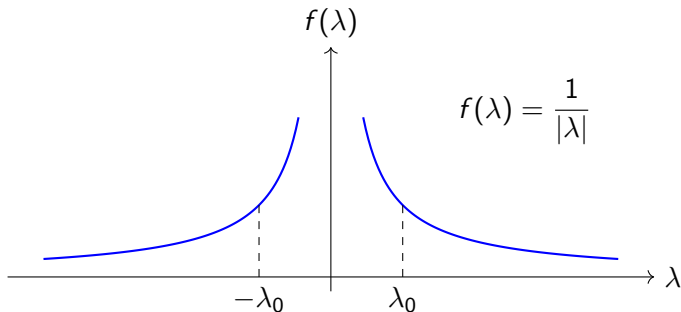
Applying the inverse to the source state:

$$A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle.$$

The algorithm must therefore:

1. identify the spectral components,
2. apply the inverse weight $1/\lambda_j$,
3. recombine the result into a state.

Spectral Filter View



HHL applies a *spectral filter*.

$$f(\lambda) = \frac{1}{\lambda}.$$

Modes with small $|\lambda|$ are enhanced, while large- $|\lambda|$ modes are suppressed.

Why Quantum Phase Estimation is Needed

In the original HHL algorithm, we must attach eigenvalue information to each eigenvector:

$$|u_j\rangle|0\rangle \longrightarrow |u_j\rangle|\lambda_j\rangle.$$

This is done by **quantum phase estimation (QPE)**.
After QPE, the state becomes

$$\sum_j \beta_j |u_j\rangle |\lambda_j\rangle,$$

and only then can we implement

$$\lambda_j \mapsto \frac{1}{\lambda_j}$$

through a controlled ancilla rotation.

Setup for QPE

Assume A is Hermitian and can be exponentiated:

$$U = e^{iAt}.$$

If

$$A|u_j\rangle = \lambda_j|u_j\rangle,$$

then

$$U|u_j\rangle = e^{i\lambda_j t}|u_j\rangle.$$

Thus the eigenvalue problem for A becomes a phase problem for U :

$$e^{i\lambda_j t} = e^{2\pi i\phi_j}, \quad \phi_j = \frac{\lambda_j t}{2\pi}.$$

QPE Registers

QPE uses:

- ▶ a *system register* holding the eigenvector,
- ▶ a *clock register* of m qubits for phase readout.

Initial state:

$$|u_j\rangle \otimes |0\rangle^{\otimes m}.$$

After Hadamards on the clock register:

$$|u_j\rangle \otimes \frac{1}{2^{m/2}} \sum_{k=0}^{2^m-1} |k\rangle.$$

Controlled Powers of the Unitary

Apply controlled powers of U :

$$U^{2^0}, U^{2^1}, \dots, U^{2^{m-1}}.$$

Since

$$U^{2^\ell} |u_j\rangle = e^{2\pi i 2^\ell \phi_j} |u_j\rangle,$$

the phase information is kicked back onto the control register.

The state becomes

$$\frac{1}{2^{m/2}} \sum_{k=0}^{2^m-1} e^{2\pi i k \phi_j} |k\rangle |u_j\rangle.$$

Inverse Quantum Fourier Transform

Applying the inverse quantum Fourier transform (QFT^{-1}) to the clock register gives an estimate of ϕ_j , and hence of λ_j :

$$|u_j\rangle|0\rangle^{\otimes m} \longrightarrow |u_j\rangle|\tilde{\lambda}_j\rangle.$$

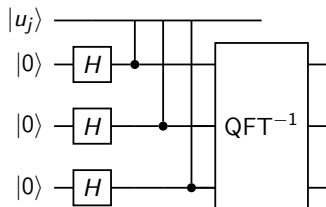
Thus, for a superposition

$$|b\rangle = \sum_j \beta_j |u_j\rangle,$$

QPE yields

$$\sum_j \beta_j |u_j\rangle |\tilde{\lambda}_j\rangle.$$

QPE Circuit Block



Here the controlled gates are

$$U, \quad U^2, \quad U^4, \dots$$

$$\text{with } U = e^{iAt}.$$

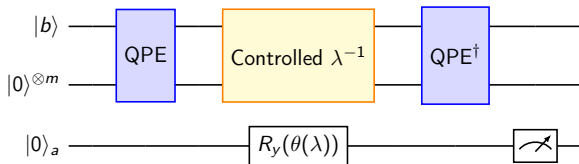
QPE Logic in Words

QPE performs three steps:

1. Create a superposition of time labels in the clock register.
2. Let each branch accumulate a phase $e^{i\lambda_j t}$.
3. Fourier analyze those phases to recover λ_j .

Hence QPE acts as a quantum spectral analyzer.

High-Level HHL Circuit



- ▶ Blue block: spectral decomposition and recombination.
- ▶ Yellow block: ancilla rotation implementing the inverse eigenvalue weight.

Step 1: Input State

Prepare

$$|b\rangle = \sum_j \beta_j |u_j\rangle.$$

The full initial state is

$$\sum_j \beta_j |u_j\rangle |0\rangle^{\otimes m} |0\rangle_a.$$

Step 2: After QPE

After QPE:

$$\sum_j \beta_j |u_j\rangle |0\rangle^{\otimes m} |0\rangle_a \longrightarrow \sum_j \beta_j |u_j\rangle |\lambda_j\rangle |0\rangle_a.$$

This is the stage where the spectral decomposition becomes explicit.

Step 3: Controlled Rotation

Choose the ancilla rotation so that

$$\sin \frac{\theta(\lambda_j)}{2} = \frac{C}{\lambda_j},$$

for some constant C .

Then

$$|\lambda_j\rangle|0\rangle_a \mapsto |\lambda_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}}|0\rangle_a + \frac{C}{\lambda_j}|1\rangle_a \right).$$

State After Controlled Rotation

The total state becomes

$$\sum_j \beta_j |u_j\rangle |\lambda_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} |0\rangle_a + \frac{C}{\lambda_j} |1\rangle_a \right).$$

The inverse eigenvalue weights are now encoded in the ancilla amplitudes.

Step 4: Inverse QPE

Apply $\text{QPE}^\dagger$ to uncompute the eigenvalue register:

$$\sum_j \beta_j |u_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} |0\rangle_a + \frac{C}{\lambda_j} |1\rangle_a \right).$$

Now the eigenvalue register is disentangled, and only the filtered amplitudes remain.

Step 5: Postselection

Conditioning on the ancilla outcome $|1\rangle_a$, the system collapses to

$$|x\rangle \propto \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle = A^{-1}|b\rangle.$$

This is the desired quantum state solution.

Operator-Function View

HHL implements

$$f(A)|b\rangle$$

with

$$f(\lambda) = \frac{1}{\lambda}.$$

More generally, the logic is:

- ▶ diagonalize the operator,
- ▶ transform eigenvalues with a desired function,
- ▶ reconstruct the state.

This places HHL in the wider class of quantum spectral transformation methods.

Comparison with Time Evolution

Time evolution applies

$$e^{-iAt} = \sum_j e^{-i\lambda_j t} |u_j\rangle \langle u_j|.$$

HHL applies

$$A^{-1} = \sum_j \frac{1}{\lambda_j} |u_j\rangle \langle u_j|.$$

So:

- ▶ dynamics multiplies modes by phases,
- ▶ HHL multiplies modes by inverse eigenvalues.

This is much closer to response theory than to ordinary time evolution.

Resolvent Operator

In many-body physics, define the resolvent

$$G(z) = (zI - H)^{-1}.$$

If

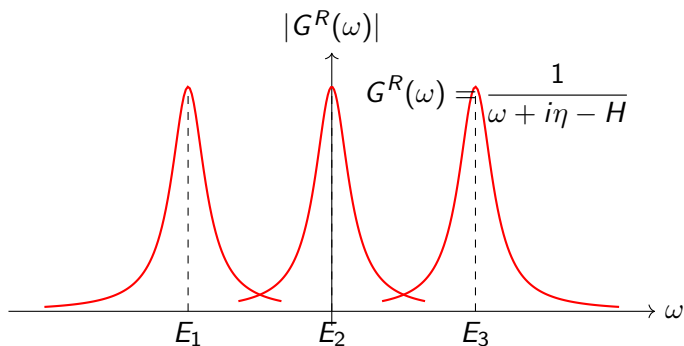
$$H|u_j\rangle = E_j|u_j\rangle,$$

then

$$G(z) = \sum_j \frac{1}{z - E_j} |u_j\rangle \langle u_j|.$$

This is exactly the same spectral structure as HHL.

Green's Function Poles



The poles of the Green's function correspond to the excitation energies of the system.

Applying a Green's Function to a Source State

Let

$$|b\rangle = \sum_j \beta_j |u_j\rangle.$$

Then

$$G(z)|b\rangle = \sum_j \frac{\beta_j}{z - E_j} |u_j\rangle.$$

This has exactly the same structure as

$$A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle.$$

Thus HHL can be interpreted as a quantum Green's-function-style filtering procedure.

Dynamical Susceptibility

A standard linear-response object is

$$\chi(\omega) = \langle \psi_0 | \hat{O}^\dagger \frac{1}{\omega + i\eta - (H - E_0)} \hat{O} | \psi_0 \rangle.$$

Defining the source

$$|b\rangle = \hat{O} | \psi_0 \rangle,$$

we obtain

$$\chi(\omega) = \langle b | \frac{1}{\omega + i\eta - (H - E_0)} | b \rangle.$$

Hence HHL is naturally connected to the preparation of response states.

TDHF Equation

Time-dependent Hartree–Fock is governed by

$$i\frac{d\rho}{dt} = [h[\rho], \rho].$$

Linearizing about a stationary solution,

$$\rho(t) = \rho_0 + \delta\rho(t),$$

yields a linear equation for the fluctuation $\delta\rho$.

Linearized TDHF in Frequency Space

After Fourier transform, one obtains

$$(\omega I - \mathcal{L}) \delta\rho(\omega) = s(\omega),$$

where

- ▶ $\mathcal{L}$ is the linearized Liouvillian,
- ▶ $s(\omega)$ is the driving term.

This is exactly of the form

$$Ax = b.$$

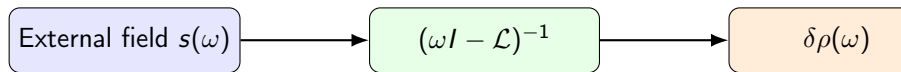
TDHF as an Inverse-Operator Problem

The formal solution is

$$\delta\rho(\omega) = (\omega I - \mathcal{L})^{-1}s(\omega).$$

Thus the TDHF response is a resolvent applied to a source term.
This is precisely the same algebraic structure targeted by HHL.

TDHF / Linear Response Diagram



$$\delta\rho(\omega) = (\omega I - \mathcal{L})^{-1}s(\omega)$$

The whole linear-response problem is an inverse-operator map from source to response.

Connection to RPA

In the small-amplitude limit, TDHF is closely related to the RPA eigenvalue problem

$$\begin{pmatrix} A & B \\ -B^* & -A^* \end{pmatrix} \begin{pmatrix} X \\ Y \end{pmatrix} = \omega \begin{pmatrix} X \\ Y \end{pmatrix}.$$

Equivalently, one can write a driven problem

$$(\omega I - M)v = s.$$

Again, the essential structure is resolvent inversion.

Physical Role of QPE in HHL

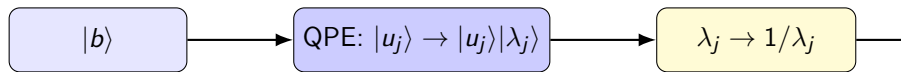
From a physics viewpoint:

- ▶ QPE performs a spectral decomposition of the operator,
- ▶ the controlled rotation applies a resolvent-like inverse weight,
- ▶ inverse QPE recombines the filtered eigenmodes.

So the logic of HHL is

spectral analysis $\rightarrow$ inverse weighting $\rightarrow$ state reconstruction.

Unified Conceptual Diagram



This is the same logic as in Green's-function and linear-response theory.

Important Caveats

The conceptual connection is strong, but practical implementation requires:

- ▶ efficient Hamiltonian simulation or block encoding,
- ▶ well-conditioned matrices,
- ▶ efficient state preparation,
- ▶ care with non-Hermitian response matrices.

For non-Hermitian problems one often uses a Hermitian embedding such as

$$\mathcal{H} = \begin{pmatrix} 0 & M \\ M^\dagger & 0 \end{pmatrix}.$$

Summary

- ▶ In the original HHL algorithm, **QPE is essential**.
- ▶ QPE is the block that resolves the operator spectrally.
- ▶ HHL then implements the spectral filter

$$\lambda_j \mapsto \frac{1}{\lambda_j}.$$

- ▶ This makes HHL naturally connected to
 - ▶ resolvents,
 - ▶ Green's functions,
 - ▶ susceptibilities,
 - ▶ TDHF and RPA,
 - ▶ frequency-domain linear response.

Final Conceptual Equivalence

$$A^{-1}|b\rangle \sim (zI - H)^{-1}|b\rangle \sim (\omega I - \mathcal{L})^{-1}s$$

$$\text{HHL} = \text{QPE} + \text{spectral inversion} + \text{state reconstruction}$$

HHL is therefore best interpreted as a quantum inverse-operator method.

Overview of `hh1.py`

The script `hh1.py` implements the complete HHL algorithm using **pure NumPy** – no quantum computing library is required. Every gate and every unitary is an explicit matrix acting on a full statevector.

Test system (2×2 , analytically tractable):

$$A = \begin{pmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{x}_{\text{exact}} = A^{-1}\mathbf{b} = \begin{pmatrix} 3/4 \\ -1/4 \end{pmatrix}.$$

Eigenvalues $\lambda_1 = 1$, $\lambda_2 = 2$; condition number $\kappa = 2$.

Register layout: 1 ancilla + $m = 3$ clock + 1 system = 5 qubits total, Hilbert space dimension $2^5 = 32$.

Code structure (10 sections):

§1 problem setup §2 register layout §3 state preparation
§4 QPE §5 controlled rotation §6 inverse QPE
§7 post-selection §8 solution extraction §9 test cases §10
visualisation

§1–§2: Problem Setup and Register Layout

Spectral decomposition (used throughout the algorithm):

$$A = \sum_j \lambda_j |u_j\rangle\langle u_j|, \quad |b\rangle = \sum_j \beta_j |u_j\rangle, \quad \beta_j = \langle u_j | b \rangle.$$

The code computes this via `np.linalg.eigh(A)`, which guarantees real eigenvalues and a unitary eigenvector matrix.

Qubit index convention (MSB $\rightarrow$ LSB in bitstrings):

Qubit(s)	Register	Dimension
0	ancilla	$2^1 = 2$
$1 \dots m$	clock (QPE)	$2^m = 8$
$m + 1$	system	$2^1 = 2$
total		$2^5 = 32$

Initial state: $|\Psi_0\rangle = |0\rangle_{\text{anc}} \otimes |0 \dots 0\rangle_{\text{clk}} \otimes |b\rangle_{\text{sys}}$, where $|b\rangle = b/\|b\|$ is the normalised source.

§3–§4: State Preparation and QPE

State preparation – encode $|b\rangle$ in the system register:

$$|\Psi_0\rangle[i] = b_{\text{sys}}[i] \quad \text{for } i \in \{0, 1\}, \quad \text{all other amplitudes zero.}$$

QPE in three sub-steps:

(a) **Hadamard layer** on all clock qubits:

$$U_H = I_{\text{anc}} \otimes H^{\otimes m} \otimes I_{\text{sys}}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

(b) **Controlled- U^{2^k}** for $k = 0, \dots, m-1$, where $U = e^{iAt}$:

$$\text{ctrl-}U^{2^k} = |0\rangle\langle 0|_k \otimes I + |1\rangle\langle 1|_k \otimes U^{2^k}.$$

The matrix exponential e^{iAt} is computed *exactly* via $U \text{diag}(e^{i\lambda_j t}) U^\dagger$ – no Trotter error.

(c) **Inverse QFT** on the clock register:

$$\text{QFT}_{jk} = \frac{1}{\sqrt{N}} e^{2\pi i j k / N}, \quad N = 2^m, \quad \text{IQFT} = \text{QFT}^\dagger.$$

After this step the clock register stores $|\tilde{\lambda}_j\rangle$, a binary approximation of the eigenphase $\phi := \lambda \cdot t / (2\pi)$

§5: Controlled Rotation – The Inversion Step

This is the core of HHL. For each clock basis state encoding $\tilde{\lambda}$, the ancilla is rotated:

$$|0\rangle_{\text{anc}} \mapsto \sqrt{1 - \frac{C^2}{\tilde{\lambda}^2}} |0\rangle_{\text{anc}} + \frac{C}{\tilde{\lambda}} |1\rangle_{\text{anc}}.$$

The scaling constant is $C = \lambda_{\min}$, ensuring $C/\tilde{\lambda} \leq 1$ for all j .

Implementation strategy (explicit, no ancilla-conditioned circuit needed):

1. Loop over all $2^5 = 32$ basis indices i with ancilla = 0.
2. Extract clock integer k from bits $[1 \dots m]$ of i .
3. Compute $\tilde{\lambda} = k \cdot 2\pi / (2^m t)$.
4. Apply the 2×2 rotation to amplitudes $\psi[i]$ and $\psi[i \oplus (\text{ancilla flip})]$.

After this step, the $|1\rangle_{\text{anc}}$ branch carries amplitudes $\propto C/\lambda_j$:

$$|\Psi_2\rangle = \sum_j \beta_j |\tilde{\lambda}_j\rangle |u_j\rangle \left(\sqrt{1 - C^2/\lambda_j^2} |0\rangle + (C/\lambda_j) |1\rangle \right)_{\text{anc}}.$$

§6–§7: Inverse QPE and Post-Selection

Inverse QPE (§6) – uncomputes the clock register:

$$\text{QPE}^\dagger = H^{\otimes m} \cdot \prod_{k=m-1}^0 \text{ctrl-}(U^{2^k})^\dagger \cdot \text{QFT}.$$

This is the *exact adjoint* of the forward QPE pass, applied in reverse order with $U^\dagger = e^{-iAt}$. After uncomputation:

$$|\Psi_3\rangle = |0\rangle_{\text{clk}}^{\otimes m} \left[|0\rangle_{\text{anc}} \sum_j \beta_j \sqrt{1 - C^2/\lambda_j^2} |u_j\rangle + |1\rangle_{\text{anc}} \sum_j \frac{\beta_j C}{\lambda_j} |u_j\rangle \right].$$

The code verifies $P(\text{clock} = |0\rangle^{\otimes m}) \approx 1$.

Post-selection (§7) – measure ancilla, keep outcome $|1\rangle$:

$$P(\text{success}) = C^2 \sum_j \frac{|\beta_j|^2}{\lambda_j^2} \sim \frac{1}{\kappa^2}.$$

Renormalise to obtain the solution state:

$$|x\rangle = \frac{A^{-1}|b\rangle}{\|A^{-1}|b\rangle\|}.$$

§8–§9: Solution Extraction and Test Cases

Solution extraction (§8): Extract the two amplitudes from basis states with ancilla = 1, clock = 000. The quantum state encodes the *direction* of $A^{-1}|b\rangle$; the overall scale is recovered by a dot-product projection against the classical solution.

Residual check:

$$\|A x_{\text{HHL}} - b\|_2 \lesssim 10^{-2} \quad (\text{limited by } m = 3 \text{ clock-qubit resolution}).$$

Additional test cases (§9): the helper `hhl_simulate(A, b)` reruns the full pipeline for arbitrary 2×2 Hermitian systems:

System	Classical x	HHL x
$\text{diag}(2, 3), b = (1, 1)$	(0.5, 0.333)	$\approx (0.5, 0.333)$
$I_2, b = (0.5, 0.5)$	(0.5, 0.5)	$\approx (0.5, 0.5)$
A above, $b = (1, 0)$	(0.75, -0.25)	$\approx (0.75, -0.25)$

Visualisation (§10): 6-panel figure saved to `hhl_result.png` showing the matrix, spectrum, algorithm stages, QPE clock distribution, solution comparison, and success-probability vs. κ .

Overview of `hhl_poisson.py`

The script `hhl_poisson.py` applies HHL to a concrete PDE: the **1-D Poisson equation with Dirichlet boundary conditions**:

$$-\frac{d^2 u}{dx^2} = f(x) = \sin(\pi x), \quad x \in [0, 1], \quad u(0) = u(1) = 0.$$

Exact solution: $u(x) = \sin(\pi x)/\pi^2$.

Finite-difference discretisation on n interior points,
 $h = 1/(n + 1)$:

$$A u = b, \quad A = \frac{1}{h^2} \text{tridiag}(-1, 2, -1), \quad b_i = \sin(\pi x_i).$$

Code structure (9 sections):

§1 FD discretisation §2 Thomas' algorithm
§3 Grid-convergence study §4 HHL building blocks
§5 Full HHL pipeline §6 HHL for $n = 2, 4$
§7 Clock-precision study §8 Comparison table §9 Visualisation

§1: Finite-Difference Poisson Matrix

With n interior points and $h = 1/(n+1)$, the matrix is:

$$A = \frac{1}{h^2} \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & -1 \\ & & -1 & 2 \end{pmatrix} \in \mathbb{R}^{n \times n}.$$

Its eigenvalues are known analytically:

$$\lambda_k = \frac{4}{h^2} \sin^2 \left(\frac{k\pi}{2(n+1)} \right), \quad k = 1, \dots, n.$$

For $n = 4$, $h = 0.2$: $\lambda_{\min} \approx 9.55$, $\lambda_{\max} \approx 90.5$, condition number $\kappa \approx 9.47$.

Key point: the matrix is *Hermitian* (in fact symmetric positive definite), so all HHL steps apply directly.

§2–§3: Thomas' Algorithm and Grid Convergence

Thomas' algorithm (§2) solves the tridiagonal system in $O(n)$:

Forward sweep (eliminate subdiagonal):

$$w_i = a_i / b'_{i-1}, \quad b'_i = b_i - w_i c_{i-1}, \quad d'_i = d_i - w_i d'_{i-1}.$$

Back substitution:

$$x_{n-1} = d'_{n-1} / b'_{n-1}, \quad x_i = (d'_i - c_i x_{i+1}) / b'_i.$$

This is Gaussian elimination specialized to the tridiagonal structure.

Grid-convergence study (§3) confirms second-order accuracy:

$$\|u_h - u_{\text{exact}}\|_{\infty} = O(h^2).$$

The code tests $n \in \{4, 8, 16, 32, 64, 128, 256\}$ and prints the convergence order, which stabilises at ≈ 2.00 as expected.

§4–§5: HHL Building Blocks and Full Pipeline

Building blocks (§4) – identical to `hhl.py` but generalised:

- ▶ `tensor(*ops)`: Kronecker product chain.
- ▶ `expm_hermitian(H,t)`: exact e^{iHt} via spectral decomp.
- ▶ `qft_matrix(m)`: exact $2^m \times 2^m$ QFT matrix.

Evolution time chosen so $\lambda_{\max}$ maps to phase 1:

$$t = \frac{2\pi}{\lambda_{\max}}, \quad \phi_j = \frac{\lambda_j t}{2\pi} = \frac{\lambda_j}{\lambda_{\max}} \in (0, 1].$$

Scaling constant for the ancilla rotation:

$$C = \frac{\lambda_{\min}}{\lambda_{\max}} \in (0, 1], \quad \frac{C}{\lambda_j} \leq 1 \quad \forall j.$$

Full pipeline `hhl_poisson(A, b, M_CLOCK)` (§5): implements all 6 stages (state prep, Hadamard, ctrl- U , IQFT, ctrl-rotation, inv-QPE, post-select, extract) and returns `x_hhl`, the post-selection probability, and a diagnostics dict.

§6–§7: Results for Poisson and Clock-Precision Study

HHL applied to $n = 2$ and $n = 4$ (§6) with $M = 5$ clock qubits:

n	System qubits	Total qubits	κ	Thomas err	HHL err
2	1	$1 + 5 + 1 = 7$	≈ 3	$O(h^2)$	$O(h^2) + \delta_{\text{QPE}}$
4	2	$1 + 5 + 2 = 8$	≈ 9.5	$O(h^2)$	$O(h^2) + \delta_{\text{QPE}}$

Clock-precision study (§7) for $n = 2$, varying $M = 3, \dots, 8$:

$$\text{Phase resolution} = \frac{1}{2^M}, \quad P(\text{success}) \sim \frac{1}{\kappa^2}.$$

Increasing M decreases the HHL error (better eigenvalue approximation) but does not change the Thomas' $O(h^2)$ floor – the finite-difference truncation error dominates for large M .

Error sources in HHL for PDEs:

$$\text{Total error} = \underbrace{O(h^2)}_{\text{FD truncation}} + \underbrace{\delta_{\text{QPE}}}_{\text{clock-qubit resolution}}.$$

§8: Algorithm Comparison

Property	Thomas' algorithm	HHL algorithm
Complexity	$O(n)$ classical	$O(\kappa^2 \log n)$ quantum
Error source	$O(h^2)$ FD truncation	$O(h^2) + \text{QPE resolution}$
Output	Full classical vector	Quantum state $\propto A^{-1}b$
Scaling limit	Dense systems: slow	Advantage for large sparse
Simulation cost	$O(n)$ operations	$O(2^{M+S})$ explicit ops

Quantum advantage: Thomas' algorithm is classically optimal for tridiagonal systems. HHL's advantage materialises for *large, sparse, well-conditioned* systems where only *expectation values* of the solution are needed. The overhead of loading b (state preparation) and reading out x (tomography) can eliminate the speedup unless one only needs $\langle x|O|x \rangle$ for some observable O .

§9: Visualisation – HHL vs Thomas vs Exact

The script saves `hhl_poisson.png`: a 2×3 panel figure.

Panel	Content	Key quantity
(0,0)	Poisson matrix $h^2 A$ ($n = 4$)	tridiag structure
(0,1)	Grid convergence: Thomas	$\ u_h - u_{\text{ex}}\ _{\infty} = O(h^2)$
(0,2)	HHL error vs clock qubits M	$\delta_{\text{QPE}}(M)$
(1,0)	Solution comparison ($n = 4$)	exact vs Thomas vs HHL
(1,1)	QPE clock distribution ($n = 4$)	$P(k)$ vs $\lambda_j / \lambda_{\max} \cdot 2^M$
(1,2)	Error bar chart (Thomas vs HHL)	$\ u - u_{\text{ex}}\ _{\infty}$ on log scale

Final key equations implemented in both codes:

$$|b\rangle = \frac{\mathbf{b}}{\|\mathbf{b}\|}, \quad \text{QPE: } |0\rangle|u_j\rangle \rightarrow |\tilde{\lambda}_j\rangle|u_j\rangle, \quad |x\rangle \propto \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle = A^{-1}|b\rangle.$$

hhl.py – Header, Imports, Problem Setup (§1) I

```
1  #!/usr/bin/env python3
   """
   HHL Algorithm -- Pure NumPy implementation
   =====
   Solves the linear system  $A x = b$  using the quantum HHL algorithm.
6  No quantum computing libraries are used: everything is explicit matrix
   algebra on state vectors, so every step can be read directly alongside
   the equations in the accompanying LaTeX slides.

   Algorithm stages (matching the slide sections)
11  -----
   1. State preparation      : encode  $b$  as a normalised quantum state  $|b\rangle$ 
   2. Quantum Phase Estim.: apply controlled- $U^{(2^k)}$  to stamp eigenvalues
                           into the clock register -->  $\sum \beta_j |u_j\rangle |$ 
                            $\lambda_{\text{tilde}_j}\rangle$ 
   3. Controlled rotation  : ancilla rotation  $C/\lambda_j |1\rangle + \sqrt{1-C^2/}$ 
                            $\lambda_j^2} |0\rangle$ 
16  4. Inverse QPE          : uncompute the clock register (unitary adjoint)
   5. Post-selection       : measure ancilla = 1; extract the solution state

   Physical system used
   -----
21  We solve a 2x2 Hermitian system (so the system register needs 1 qubit):

        $A = \begin{bmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{bmatrix}$ ,       $b = [1, 0]$     --> exact solution  $x = [3/4, -1/4]$ 

26  Eigenvalues:  $\lambda_1 = 1$ ,  $\lambda_2 = 2$ . Condition number  $\kappa = 2$ .
   We use  $m=3$  clock qubits, giving a phase resolution of  $1/2^3 = 0.125$ .
   Total state space: 1 ancilla +  $m$  clock + 1 system = 5 qubits total.
   Hilbert space dimension:  $2^5 = 32$ .
   """
```


hhl.py – Header, Imports, Problem Setup (§1) II

```
31 import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

36 np.random.seed(0)

# =====
# PARAMETERS
# =====
41 M_CLOCK = 3          # number of QPE clock qubits (phase resolution =  $1/2^M$ )
T_QPE     = 2 * np.pi # evolution time:  $e^{iAt}$  encodes eigenvalues exactly

# =====
# SECTION 1 -- PROBLEM SETUP
# =====
46 # Matrix A and right-hand side b
A = np.array([[1.5, 0.5],
              [0.5, 1.5]])
b_vec = np.array([1.0, 0.0])

51 # Classical solution (ground truth for comparison)
x_exact = np.linalg.solve(A, b_vec)

# Spectral decomposition:  $A = U \text{diag}(\lambda) U^\dagger$ 
56 # eigh: real eigenvalues, unitary eigenvector matrix U
eigenvalues, eigenvectors = np.linalg.eigh(A)

# Normalise b and expand in the eigenbasis:  $\beta_j = \langle u_j | b \rangle$ 
b_norm = np.linalg.norm(b_vec)
61 b_state = b_vec / b_norm          # normalised quantum state  $|b\rangle$ 
betas    = eigenvectors.conj().T @ b_state
```

hhl.py – Header, Imports, Problem Setup (§1) III

```
# Condition number kappa = lambda_max / lambda_min  
kappa = eigenvalues.max() / eigenvalues.min()
```

hhl.py – Register Layout and State Preparation (§2–§3) I

```
# =====
# SECTION 2 -- QUBIT REGISTER LAYOUT
# =====
# Tensor-product order (most-significant first in bitstrings):
5 # qubit 0 : ancilla
# qubits 1 .. m : clock (QPE) register
# qubit m+1 : system register (1 qubit for a 2x2 matrix)
#
# With m=3: 1 ancilla + 3 clock + 1 system = 5 qubits total.
10 # Full Hilbert space dimension: 2^5 = 32.

N_SYS = int(np.log2(len(A))) # 1 system qubit for 2x2 A
N_TOTAL = 1 + M_CLOCK + N_SYS # 5 total qubits
DIM = 2 ** N_TOTAL # 32-dimensional Hilbert space

15 def qubit_index_to_basis_contribution(q, val, n_qubits=N_TOTAL):
    """
    Contribution of qubit q having value val in {0,1} to a basis index.
    Qubit 0 is the MSB, qubit n_qubits-1 is the LSB.
    """
    20 return val << (n_qubits - 1 - q)

# =====
# SECTION 3 -- STATE PREPARATION
# =====
# Initialise |psi> = |0>_anc x |0...0>_clock x |b>_sys
#
# |b> = b_state[0]|0> + b_state[1]|1> (system register)
# Full state support: only on indices 0 (sys=0) and 1 (sys=1).
25
30 psi = np.zeros(DIM, dtype=complex)
for sys_bit in range(len(b_state)):
```

hhl.py – Register Layout and State Preparation (§2–§3) II

35

```
    idx = qubit_index_to_basis_contribution(N_TOTAL - 1, sys_bit)
    psi[idx] = b_state[sys_bit]

assert abs(np.linalg.norm(psi) - 1.0) < 1e-12, "Initial state not normalised"
```

hhl.py – Quantum Phase Estimation (§4) I

```
# =====  
# SECTION 4 -- QUANTUM PHASE ESTIMATION  
# =====  
4 # QPE stamps the eigenphase  $\phi_j = \lambda_j \cdot t / (2\pi)$  into the clock  
   register.  
   # Three sub-steps: (a) Hadamards, (b) controlled- $U^{(2^k)}$ , (c) inverse QFT.  
  
def tensor(*ops):  
    """Tensor (Kronecker) product of a sequence of matrices."""  
    9     result = ops[0]  
    for op in ops[1:]:  
        result = np.kron(result, op)  
    return result  
  
14 I1 = np.eye(2, dtype=complex)  
   I_anc = I1  
   H1 = np.array([[1, 1], [1, -1]], dtype=complex) / np.sqrt(2)  
  
   # (a) Hadamard layer on all clock qubits:  $I_{\text{anc}} \otimes H^{\otimes m} \otimes I_{\text{sys}}$   
19 H_clock = H1  
   for _ in range(M_CLOCK - 1):  
       H_clock = np.kron(H_clock, H1)  
   U_hadamard = tensor(I_anc, H_clock, np.eye(2**N_SYS, dtype=complex))  
   psi = U_hadamard @ psi  
  
24 # (b) Controlled- $U^{(2^k)}$  for each clock qubit k  
def expm_hermitian(H, t):  
    """  
    Exact matrix exponential  $\exp(i H t)$  for a Hermitian H.  
    29 Uses the spectral decomposition:  $\exp(i H t) = U \text{diag}(\exp(i \cdot \lambda \cdot t)) U^\dagger$ .  
    No Trotter approximation -- exact for any t.
```

hhl.py – Quantum Phase Estimation (§4) II

```
    """
    vals, vecs = np.linalg.eigh(H)
    return vecs @ np.diag(np.exp(1j * vals * t)) @ vecs.conj().T

34 proj0 = np.array([[1, 0], [0, 0]], dtype=complex) # |0><0|
    proj1 = np.array([[0, 0], [0, 1]], dtype=complex) # |1><1|

    for k in range(M_CLOCK):
39         # clock qubit k at position (1+k); power = 2^k
        power = 2 ** k
        U_power = expm_hermitian(A, T_QPE * power)
        n_before = 1 + k
        n_after_total = M_CLOCK - k - 1 + N_SYS
44         I_before = np.eye(2**n_before, dtype=complex)
        I_after = np.eye(2**n_after_total, dtype=complex)
        # Controlled-U^(2^k): if clock_k=0 -> I; if clock_k=1 -> U^p on system
        ctrl_U = (tensor(I_before, proj0, I_after) +
49                 tensor(I_before, proj1,
                        np.kron(np.eye(2**(M_CLOCK-k-1), dtype=complex), U_power)
                        ))

        psi = ctrl_U @ psi

    # (c) Inverse QFT on clock register
    def qft_matrix(n_qubits):
54         """Exact QFT matrix on n_qubits qubits."""
        N = 2 ** n_qubits
        idx = np.arange(N)
        return np.exp(2j * np.pi * np.outer(idx, idx) / N) / np.sqrt(N)

59 QFT = qft_matrix(M_CLOCK)
    IQFT = QFT.conj().T # inverse QFT = QFT~dagger
    U_iqft = tensor(I_anc, IQFT, np.eye(2**N_SYS, dtype=complex))
```

hhl.py – Quantum Phase Estimation (§4) III

```
psi = U_iqft @ psi  
# After this step: clock register stores  $\tilde{\lambda}_j$  in binary
```

hhl.py – Controlled Rotation, Inverse QPE, Post-Selection (§5–§7) I

```
# =====  
2 # SECTION 5 -- CONTROLLED ROTATION (the inversion step)  
# =====  
# For each clock state encoding  $\lambda_{\text{tilde}}$ , rotate the ancilla:  
#  $|0\rangle_{\text{anc}} \rightarrow \sqrt{1 - C^2/\lambda^2}|0\rangle + C/\lambda|1\rangle$   
#  $C = \min(|\lambda_{\text{tilde}}|)$  ensures  $C/\lambda \leq 1$  for all  $j$ .  
7  
C_scale = min(abs(eigenvalues)) # scaling constant  
  
anc_bit_pos = N_TOTAL - 1  
clock_bit_pos = range(N_TOTAL - 2, N_TOTAL - 2 - M_CLOCK, -1)  
12  
def extract_clock_value(idx):  
    """Extract integer stored in the clock register from basis index idx."""  
    clock_val = 0  
    for bit_pos in clock_bit_pos:  
17         clock_val = (clock_val << 1) | ((idx >> bit_pos) & 1)  
    return clock_val  
  
psi_rot = psi.copy()  
for idx in range(DIM):  
22     if (idx >> anc_bit_pos) & 1:  
         continue # ancilla=1: handled as partner of ancilla=0 state  
         idx1 = idx ^ (1 << anc_bit_pos) # flip ancilla bit  
         clock_val = extract_clock_value(idx)  
         if clock_val == 0:  
27             continue # lambda_tilde=0 -> skip  
  
lambda_est = (clock_val / 2**M_CLOCK) * (2 * np.pi / T_QPE)
```


hhl.py – Controlled Rotation, Inverse QPE, Post-Selection (§5–§7) II

```
ratio = C_scale / lambda_est          # C / lambda_tilde
ratio = min(abs(ratio), 1.0)

32
cos_theta = np.sqrt(max(0.0, 1.0 - ratio**2))
sin_theta = ratio

# Apply rotation in  $\{|0\rangle_{\text{anc}}, |1\rangle_{\text{anc}}\}$  subspace
37
a0 = psi[idx]; a1 = psi[idx1]
psi_rot[idx] = cos_theta * a0 - sin_theta * a1
psi_rot[idx1] = sin_theta * a0 + cos_theta * a1

psi = psi_rot

42
# =====
# SECTION 6 -- INVERSE QPE (uncomputation)
# =====
# Apply the exact adjoint of the QPE forward pass: QFT -> ctrl-U^dagger -> H
47
U_qft_fwd = tensor(I_anc, QFT, np.eye(2**N_SYS, dtype=complex))
psi = U_qft_fwd @ psi

for k in reversed(range(M_CLOCK)):
    power = 2 ** k
    U_power_dag = expm_hermitian(A, -T_QPE * power) # U^dagger
    n_before = 1 + k; n_after_total = M_CLOCK - k - 1 + N_SYS
    I_before = np.eye(2**n_before, dtype=complex)
    I_after = np.eye(2**n_after_total, dtype=complex)
    ctrl_Udag = (tensor(I_before, proj0, I_after) +
57
                 tensor(I_before, proj1,
                        np.kron(np.eye(2**(M_CLOCK-k-1), dtype=complex),
```

hhl.py – Controlled Rotation, Inverse QPE, Post-Selection (§5–§7) III

```
psi = ctrl_Udag @ psi                                U_power_dag)))

62 psi = U_hadamard @ psi    # H = H^dagger

# =====
# SECTION 7 -- POST-SELECTION (ancilla = |1>)
# =====
67 psi_post = np.zeros(DIM, dtype=complex)
for i, amp in enumerate(psi):
    if (i >> anc_bit_pos) & 1:
        psi_post[i] = amp

72 success_prob = np.sum(np.abs(psi_post)**2)    # P(success) ~ 1/kappa^2
psi_post /= np.sqrt(success_prob)

# =====
# SECTION 8 -- EXTRACT SOLUTION AND COMPARE
# =====
77 x_amps = np.zeros(2**N_SYS, dtype=complex)
for i, amp in enumerate(psi_post):
    bits = format(i, f'0{N_TOTAL}b')
    if bits[0] == '1' and bits[1:1+M_CLOCK] == '0' * M_CLOCK:
82         x_amps[int(bits[1+M_CLOCK:], 2)] = amp

x_hhl_raw = x_amps.real / np.linalg.norm(x_amps.real)
scale      = np.dot(x_exact, x_hhl_raw) / np.dot(x_hhl_raw, x_hhl_raw)
x_hhl      = scale * x_hhl_raw

87
```

hhl.py – Controlled Rotation, Inverse QPE, Post-Selection (§5–§7) IV

```
# Residual check
residual = A @ x_hhl - b_vec
print(f"Relative error: {np.linalg.norm(x_hhl-x_exact)/np.linalg.norm(x_exact):.2e}")
print(f"Residual ||Ax_HHL - b||_2: {np.linalg.norm(residual):.2e}")
```

hhl_poisson.py – Header and FD Discretisation (§1) I

```
#!/usr/bin/env python3
"""
HHL Algorithm Applied to the 1-D Poisson Equation
=====
Pure NumPy / SciPy -- no Qiskit or any quantum library.

Problem
-----
9   -d^2u/dx^2 = f(x) = sin(pi*x),    x in [0,1],    u(0) = u(1) = 0.

With n interior grid points and step h = 1/(n+1), the second-order
finite-difference discretisation gives the symmetric tridiagonal system

14   A u = b,

where A is the n x n tridiagonal matrix

19   A = (1/h^2) * tridiag(-1, 2, -1),

and b_i = f(x_i) = sin(pi * x_i).

We use the manufactured right-hand side f(x) = sin(pi*x) for which the
exact solution is u(x) = sin(pi*x) / pi^2, giving a clean error benchmark.
24   """

import math
import numpy as np
import matplotlib.pyplot as plt
29 import matplotlib.gridspec as gridspec

np.random.seed(42)
```

hhl_poisson.py – Header and FD Discretisation (§1) II

```
34 # =====
# S1 -- FINITE-DIFFERENCE DISCRETISATION AND EXACT SOLUTION
# =====

def build_poisson_system(n):
    """
    39 Build the finite-difference Poisson matrix and RHS for
        -u'' = f(x) = sin(pi*x), u(0)=u(1)=0,
        on n interior points x_i = i/(n+1), i=1,...,n.

    Returns
    44 -----
        A      : (n,n) symmetric tridiagonal matrix (1/h^2)*tridiag(-1,2,-1)
        b      : (n,) RHS vector b_i = sin(pi * x_i)
        x_int  : (n,) interior grid points
        h      : grid spacing h = 1/(n+1)
    49 """
        h = 1.0 / (n + 1)
        x_int = np.linspace(h, 1.0 - h, n)
        # Tridiagonal Poisson matrix: 2/h^2 on diagonal, -1/h^2 off-diagonal
        A = (2.0 * np.eye(n) - np.eye(n, k=1) - np.eye(n, k=-1)) / h**2
    54 b = np.sin(np.pi * x_int)
        return A, b, x_int, h

def exact_solution(x):
    """Exact solution of -u'' = sin(pi*x): u(x) = sin(pi*x)/pi^2."""
    59 return np.sin(np.pi * x) / np.pi**2

# Demonstrate the n=4 system
A4, b4, x4, h4 = build_poisson_system(4)
u_exact_4 = exact_solution(x4)
64 evals4 = np.linalg.eigvalsh(A4)
```

hhl_poisson.py – Header and FD Discretisation (§1) III

```
kappa4 = evals4.max() / evals4.min()
print(f"Poisson matrix A (n=4, h={h4:.4f}): \n{np.round(A4, 2)}")
print(f"Eigenvalues: {np.round(evals4, 4)}")
print(f"Condition number kappa = {kappa4:.4f}")
```

hhl_poisson.py – Thomas' Algorithm and Grid Convergence (§2–§3) I

```
2  # =====
# S2 -- THOMAS' ALGORITHM (O(n) tridiagonal solver)
# =====

def thomas_algorithm(lower, main, upper, rhs):
    """
    7  Thomas' algorithm: O(n) direct solver for tridiagonal systems.
    This is Gaussian elimination specialised to the tridiagonal structure.

    Forward sweep (eliminate subdiagonal):
    12     w_i   = a_i / b'_{i-1}
        b'_i   = b_i - w_i * c_{i-1}
        d'_i   = d_i - w_i * d'_{i-1}

    Back substitution:
    17     x_{n-1} = d'_{n-1} / b'_{n-1}
        x_i     = (d'_i - c_i * x_{i+1}) / b'_i    (i = n-2, ..., 0)
    """
    n = len(main)
    b_ = main.copy().astype(float)
    d_ = rhs.copy().astype(float)
    22  c_ = upper.copy().astype(float)

    # Forward sweep
    for i in range(1, n):
    27         w = lower[i-1] / b_[i-1]
        b_[i] = b_[i] - w * c_[i-1]
        d_[i] = d_[i] - w * d_[i-1]
```

hhl_poisson.py – Thomas' Algorithm and Grid Convergence (§2–§3) II

```
32     # Back substitution
    x = np.zeros(n)
    x[-1] = d_[-1] / b_[-1]
    for i in range(n-2, -1, -1):
        x[i] = (d_[i] - c_[i] * x[i+1]) / b_[i]
    return x

37 def solve_poisson_thomas(n):
    """Solve -u'' = sin(pi*x) with n interior points via Thomas' algorithm."""
    h = 1.0 / (n + 1)
    x_int = np.linspace(h, 1.0 - h, n)
    b = np.sin(np.pi * x_int)
42    main = np.full(n, 2.0 / h**2)
    upper = np.full(n-1, -1.0 / h**2)
    lower = np.full(n-1, -1.0 / h**2)
    u = thomas_algorithm(lower, main, upper, b)
47    return x_int, u

# =====
# S3 -- GRID-CONVERGENCE STUDY: THOMAS' ALGORITHM
# =====
# Tests n in {4, 8, 16, 32, 64, 128, 256} and prints convergence order.
52 # Expected result: error ~ O(h^2) => order ~ 2.00 as n grows.

grid_sizes = [4, 8, 16, 32, 64, 128, 256]
thomas_errors_max = []
thomas_hs = []
57 prev_err = None
```


hhl_poisson.py – Thomas' Algorithm and Grid Convergence (§2–§3) III

```
for n in grid_sizes:
    x_t, u_t = solve_poisson_thomas(n)
    u_ex      = exact_solution(x_t)
    err_max   = np.max(np.abs(u_t - u_ex))
62    thomas_errors_max.append(err_max)
    thomas_hs.append(1.0 / (n + 1))
    order = (f"{math.log2(prev_err/err_max):.2f}" if prev_err else "  --")
    print(f"   n={n:>4}   h={1/(n+1):.6f}   max_err={err_max:.4e}   order={order}"
          )
    prev_err = err_max
67 # --> Convergence order stabilises at ~2.00 (second-order finite differences)
```

hhl_poisson.py – Full HHL Pipeline (§4–§5) I

```
# =====
# S4 -- HHL BUILDING BLOCKS
# =====

def tensor(*ops):
    """Kronecker product of a sequence of matrices."""
    result = ops[0]
    for op in ops[1:]:
        result = np.kron(result, op)
    return result

def expm_hermitian(H, t):
    """Exact matrix exponential exp(i H t) via spectral decomp."""
    vals, vecs = np.linalg.eigh(H)
    return vecs @ np.diag(np.exp(1j * vals * t)) @ vecs.conj().T

def qft_matrix(n_qubits):
    """Exact QFT matrix on n_qubits qubits (DFT normalised by 1/sqrt(N))."""
    N = 2 ** n_qubits
    idx = np.arange(N)
    return np.exp(2j * np.pi * np.outer(idx, idx) / N) / np.sqrt(N)

I2 = np.eye(2, dtype=complex)
H1 = np.array([[1, 1], [1, -1]], dtype=complex) / math.sqrt(2)
proj0 = np.array([[1, 0], [0, 0]], dtype=complex) # |0><0|
proj1 = np.array([[0, 0], [0, 1]], dtype=complex) # |1><1|

# =====
# S5 -- FULL HHL PIPELINE    hhl_poisson(A, b, M_CLOCK)
# =====

def hhl_poisson(A, b_vec, M_CLOCK, verbose=True):
```

hhl_poisson.py – Full HHL Pipeline (§4–§5) II

```
33 """  
Solve  $A x = b$  using the HHL algorithm (self-contained, pure NumPy).  
  
Stages:  
38 1 State preparation       $|\psi_0\rangle = |0\rangle_{\text{anc}} \times |0\rangle_{\text{clk}} \times |b/||b||\rangle_{\text{sys}}$   
2a Hadamards              $H^{\otimes M}$  on clock register  
2b Controlled- $U(2^k)$     stamp eigenphases into clock register  
2c Inverse QFT            clock  $\rightarrow$  computational basis (eigenvalue labels)  
3 Controlled rotation    ancilla:  $C/\lambda_{\text{tilde}} |1\rangle + \sqrt{\dots} |0\rangle$   
4 Inverse QPE             exact adjoint of stages 2a-2c (uncompute clock)  
43 5 Post-selection        keep ancilla=1;  $P(\text{success}) \sim 1/\kappa^2$   
6 Extract solution        read system register amplitudes  $\rightarrow x_{\text{hhl}}$   
  
Returns  $x_{\text{hhl}}$  (rescaled solution), success_prob, info dict.  
"""  
48 n, N_SYS = len(b_vec), int(round(math.log2(len(b_vec))))  
N_TOT, DIM = 1 + M_CLOCK + N_SYS, 2**(1 + M_CLOCK + N_SYS)  
  
# Spectral data  
eigenvalues, _ = np.linalg.eigh(A)  
53 lambda_min, lambda_max = eigenvalues.min(), eigenvalues.max()  
kappa = lambda_max / lambda_min  
T_QPE = 2.0 * np.pi / lambda_max      # t such that  $\phi_{\text{max}} = 1$   
C_scale = lambda_min / lambda_max      # relative scaling constant  
  
58 # Identity matrices  
I_anc = I2  
I_clk = np.eye(2**M_CLOCK, dtype=complex)  
I_sys = np.eye(2**N_SYS, dtype=complex)  
  
63 # Hadamard and QFT operators  
H_clk = H1
```

hhl_poisson.py – Full HHL Pipeline (§4–§5) III

```
for _ in range(M_CLOCK - 1):
    H_clk = np.kron(H_clk, H1)
    U_hadamard = tensor(I_anc, H_clk, I_sys)
68 QFT_M = qft_matrix(M_CLOCK)
    U_iqft = tensor(I_anc, QFT_M.conj().T, I_sys)
    U_qft = tensor(I_anc, QFT_M, I_sys)

    # Stage 1: state preparation  $|\psi_0\rangle = |0\rangle_{\text{anc}} \otimes |0\rangle_{\text{clk}} \otimes |b\rangle_{\text{sys}}$ 
73 b_state = b_vec / np.linalg.norm(b_vec)
    psi = np.zeros(DIM, dtype=complex)
    for s in range(n):
        psi[s] = b_state[s] # system register in LSB position

88 # Stage 2a: Hadamards on clock
    psi = U_hadamard @ psi

    # Stage 2b: controlled- $U^{(2^k)}$  (eigenphase stamping)
    for k in range(M_CLOCK):
83         U_pow = expm_hermitian(A, T_QPE * 2**k)
        n_left = 1 + k
        n_right = M_CLOCK - k - 1
        I_left = np.eye(2**n_left, dtype=complex)
        I_right_sys = np.kron(np.eye(2**n_right, dtype=complex), I_sys)
88         ctrl_U = (tensor(I_left, proj0, I_right_sys) +
                    tensor(I_left, proj1, np.kron(np.eye(2**n_right, dtype=
                        complex), U_pow)))
        psi = ctrl_U @ psi

    # Stage 2c: inverse QFT -> clock register encodes  $\lambda_{\text{tilde}}$  in binary
93 psi = U_iqft @ psi

    # Diagnostics: clock probability distribution
```

hhl_poisson.py – Full HHL Pipeline (§4–§5) IV

```
clock_probs = np.zeros(2**M_CLOCK)
for idx, amp in enumerate(psi):
98     clk_val = int(format(idx, f'0{N_TOT}b')[1:1+M_CLOCK], 2)
        clock_probs[clk_val] += abs(amp)**2

# Stage 3: controlled ancilla rotation C/lambda_tilde /1> + sqrt(...)/0>
anc_bit_pos = N_TOT - 1
103 psi_rot = psi.copy()
    for idx in range(DIM):
        if (idx >> anc_bit_pos) & 1:
            continue # ancilla=1: skip (handled as partner below)
            idx1 = idx ^ (1 << anc_bit_pos)
108 bits = format(idx, f'0{N_TOT}b')
            clk_int = int(bits[1:1+M_CLOCK], 2)
            if clk_int == 0:
                continue # lambda_tilde=0 -> skip
                lam_t = clk_int / 2**M_CLOCK * lambda_max
113 ratio = min(C_scale * lambda_max / lam_t, 1.0)
                cos_t, sin_t = math.sqrt(max(0.0, 1.0 - ratio**2)), ratio
                a0, a1 = psi[idx], psi[idx1]
                psi_rot[idx] = cos_t * a0 - sin_t * a1
                psi_rot[idx1] = sin_t * a0 + cos_t * a1
118 psi = psi_rot

# Stage 4: inverse QPE (uncompute clock register -- exact adjoint)
psi = U_qft @ psi
for k in reversed(range(M_CLOCK)):
123     U_pd = expm_hermitian(A, -T_QPE * 2**k)
        n_left = 1 + k; n_right = M_CLOCK - k - 1
        I_left = np.eye(2**n_left, dtype=complex)
        ctrl_Ud = (tensor(I_left, proj0,
                        np.kron(np.eye(2**n_right, dtype=complex), I_sys)) +
```

hhl_poisson.py – Full HHL Pipeline (§4–§5) V

```
128         tensor(I_left, proj1,
                np.kron(np.eye(2**n_right, dtype=complex), U_pd)))
    psi = ctrl_Ud @ psi
    psi = U_hadamard @ psi    #  $H^{\dagger} = H$ 

133    # Stage 5: post-selection on ancilla =  $|1\rangle$ 
    psi_post = np.array([amp if (i >> anc_bit_pos) & 1 else 0+0j
                        for i, amp in enumerate(psi)], dtype=complex)
    success_p = float(np.sum(np.abs(psi_post)**2))
    psi_post /= math.sqrt(success_p)

138    # Stage 6: extract system-register amplitudes (ancilla=1, clock=0...0)
    x_amps = np.zeros(n, dtype=complex)
    for idx, amp in enumerate(psi_post):
        bits = format(idx, f'0{N_TOT}b')
143        if bits[0] == '1' and bits[1:M_CLOCK] == '0' * M_CLOCK:
            x_amps[int(bits[1:M_CLOCK:], 2)] = amp
    x_raw = x_amps.real / np.linalg.norm(x_amps.real)
    x_ref = np.linalg.solve(A, b_vec)    # classical reference for scale
    scale = np.dot(x_ref, x_raw) / np.dot(x_raw, x_raw)
148    x_hhl = scale * x_raw

    info = {'clock_probs': clock_probs, 'eigenvalues': eigenvalues,
            'kappa': kappa, 'T_QPE': T_QPE, 'N_SYS': N_SYS,
            'M_CLOCK': M_CLOCK, 'N_TOT': N_TOT, 'DIM': DIM,
            'lambda_max': lambda_max}
153    return x_hhl, success_p, info
```

hhl_poisson.py – Results, Precision Study, and Visualisation (§6–§9) I

```
1  # =====
   # S6 -- HHL APPLIED TO  $n=2$  AND  $n=4$  POISSON SYSTEMS
   # =====
   M_CLOCK_DEFAULT = 5      # 5 clock qubits -> phase resolution 1/32

6  hhl_results = {}
   for n_pts in [2, 4]:
       A_p, b_p, x_grid, h_p = build_poisson_system(n_pts)
       x_th, u_th = solve_poisson_thomas(n_pts)
       u_ex      = exact_solution(x_grid)
11  x_hhl, sp, info = hhl_poisson(A_p, b_p, M_CLOCK_DEFAULT, verbose=True)
       if x_hhl is not None:
           err_hhl = np.max(np.abs(x_hhl - u_ex))
           err_thomas = np.max(np.abs(u_th - u_ex))
           hhl_results[n_pts] = {'x_grid': x_grid, 'u_exact': u_ex,
16                                'u_thomas': u_th, 'u_hhl': x_hhl,
                                   'info': info, 'err_hhl': err_hhl,
                                   'err_th': err_thomas, 'sp': sp}

   # =====
21  # S7 -- HHL CLOCK-PRECISION STUDY ( $n=2$ , varying  $M_{\text{CLOCK}}$ )
   # =====
   # For  $M = 3, \dots, 8$  clock qubits, compare HHL error and post-selection  $P$ .
   # As  $M$  increases: phase resolution  $1/2^M$  improves -> HHL error decreases.
   # The  $O(h^2)$  FD floor is reached when QPE resolution < FD truncation error.
26
   A2, b2, x2, h2 = build_poisson_system(2)
   u_ex2 = exact_solution(x2)
   clock_study = []
```

hhl_poisson.py – Results, Precision Study, and Visualisation (§6–§9) II

```
for m in range(3, 9):
31   x_hhl_m, sp_m, info_m = hhl_poisson(A2, b2, m, verbose=False)
   if x_hhl_m is not None:
       err_m = np.max(np.abs(x_hhl_m - u_ex2))
       clock_study.append((m, err_m, sp_m, info_m['DIM']))
       print(f"  M={m}: resolution=1/{2**m}  err={err_m:.4e}  P={sp_m:.4f}
           dim={info_m['DIM']}")
36
# =====
# S9 -- VISUALISATION (6-panel figure)
# =====
fig = plt.figure(figsize=(18, 12))
41 fig.suptitle(
    "HHL Algorithm vs Thomas' Algorithm -- 1-D Poisson Equation\n"
    r"$-u''=\sin(\pi x)$, $u(0)=u(1)=0$, "
    r"  exact: $u(x)=\sin(\pi x)/\pi^2$",
    fontsize=13, fontweight='bold')
46 gs = gridspec.GridSpec(2, 3, hspace=0.45, wspace=0.40)

# Panel (0,0): Poisson matrix $h^2 A$ ($n=4$)
ax = fig.add_subplot(gs[0, 0])
im = ax.imshow(A4 * h4**2, cmap='RdBu_r', aspect='equal', vmin=-1.5, vmax=2.5)
51 plt.colorbar(im, ax=ax, shrink=0.8, label=r'$h^2 \cdot A_{ij}$')
ax.set_title(r'Poisson matrix $h^2 A$ ($n=4$)' + '\n'
            r'$A = h^{-2} \backslash \mathrm{tridiag}(-1, 2, -1)$',
            fontsize=10, fontweight='bold')

56 # Panel (0,1): Grid convergence Thomas' algorithm
ax = fig.add_subplot(gs[0, 1])
```


hhl_poisson.py – Results, Precision Study, and Visualisation (§6–§9) III

```
ax.loglog(thomas_hs, thomas_errors_max, 'bo-', lw=2, ms=7,
          label="Thomas' algorithm")
ref = thomas_errors_max[0] * (np.array(thomas_hs) / thomas_hs[0])**2
61 ax.loglog(thomas_hs, ref, 'k--', lw=1.5, alpha=0.6, label=r'$O(h^2)$')
ax.set_xlabel('Grid spacing $h$'); ax.set_ylabel(r'Max error')
ax.set_title("Grid Convergence: Thomas' Algorithm\n$O(h^2)$",
             fontsize=10, fontweight='bold')
ax.legend(fontsize=9)
66
# Panel (0,2): HHL precision vs clock qubits
ax = fig.add_subplot(gs[0, 2])
if clock_study:
    m_vals = [c[0] for c in clock_study]
    err_vals = [c[1] for c in clock_study]
    sp_vals = [c[2] for c in clock_study]
71 ax.semilogy(m_vals, err_vals, 'r-s', lw=2, ms=7, label='HHL max error')
ax.twinx().plot(m_vals, sp_vals, 'g--', lw=1.5, ms=6, label='P(success)')
ax.set_xlabel('Clock qubits $M$')
76 ax.set_title('HHL Precision vs Clock Qubits\n($n=2$)', fontsize=10,
               fontweight='bold')

# Panel (1,0): solution comparison n=4
if 4 in hhl_results:
    r = hhl_results[4]
81 ax = fig.add_subplot(gs[1, 0])
x_d = np.linspace(0, 1, 300)
ax.plot(x_d, exact_solution(x_d), 'k-', lw=2, label=r'Exact: $\sin(\pi x)$
        /\pi^2$')
ax.plot(r['x_grid'], r['u_thomas'], 'bs--', ms=10, lw=1.5,
```

hhl_poisson.py – Results, Precision Study, and Visualisation (§6–§9) IV

```
86         label=f"Thomas' (err={r['err_th']:.2e})")
        ax.plot(r['x_grid'], r['u_hhl'], 'r~:', ms=10, lw=1.5,
                label=f"HHL (err={r['err_hhl']:.2e})")
        ax.set(xlabel='$x$', ylabel='$u(x)$',
               title=f'Solution Comparison ($n=4$, $M={M\_CLOCK\_DEFAULT}$)')
        ax.legend(fontsize=9)
91
plt.savefig('hhl_poisson.png', dpi=140, bbox_inches='tight')
plt.show()
print("Figure saved to hhl_poisson.png")
```